Citi Bike Machine Learning Project & Databricks Plan

CUNY 622 Final Project | September 30, 2026

Recommendation: Forecast station activity and identify transportation-access gaps. This offers a clear machine-learning task and recommendations relevant to neighborhoods with limited transportation options.

This report preserves the analysis and implementation plan developed from the two attached CSVs. The full Mac folder at `/Volumes/BigRaid/RandyFiles/Cuny/622/FinalProject` was not accessible in this session; its size and completeness still need to be inventoried.

What the samples show

Finding	December 2013	August 2026, part 1
Trips	443,966	1,000,000
Actual start-date coverage	December 1-31	July 31-August 14
Distinct starting stations	329	2,308
Median trip duration	8.9 minutes	9.6 minutes
Membership categories	Subscriber / Customer	Member / Casual
Bike type	Not provided	74.0% electric; 26.0% classic
Rider demographics	Birth year and gender	Not provided
Trips exceeding 24 hours	74	15

• **August is incomplete.** Citi Bike splits months exceeding one million trips across CSVs. All parts are needed for monthly totals. The attached rows are not chronologically sorted. [1]

• **Coverage changed substantially.** Comparing these samples mixes seasonality, station expansion, membership changes, and electric-bike adoption.

• **Missing-value conventions differ.** The older file has 19,379 birth years recorded as `\N`, and numeric birth years extend to 1899. The modern file has 425 missing start-station IDs and 332 missing end-station IDs.

• **Trips record completed travel.** They do not reveal failed attempts to find a bike, rider residence, actual routes, or the transportation mode replaced.

Possible machine-learning projects

1. Station demand forecasting with equity analysis

Task: Predict station departures and arrivals during the next hour. **Combine:** Weather, Census characteristics, transit proximity; station availability as an extension.
Benefit: Prioritize service improvements in transportation-constrained neighborhoods. **Limit:** Completed rides measure observed activity, not all unmet demand.

2. Bike-share access and station expansion

Task: Predict activity at candidate station locations. **Combine:** Census tracts, vehicle availability, population, NYCHA locations, and subway stations.
Benefit: Identify neighborhoods that might benefit from new stations. **Limit:** Training on existing stations can reproduce existing deployment bias.

3. Bicycle infrastructure investment priorities

Task: Predict future cyclist-injury crash counts by area; combine with activity and infrastructure gaps. **Combine:** NYPD collisions, DOT bicycle routes, and weather.
Benefit: Support investigation of areas needing safer infrastructure. **Limit:** Endpoints do not reveal routes or total bicycle exposure.

4. First/last-mile transit connectivity

Task: Predict bike activity near subway stations and identify complementary patterns. **Combine:** MTA hourly ridership, station locations, and weather.
Benefit: Improve connections to public transit. **Limit:** Nearby activity cannot prove individual riders transferred.

5. Electric-bike mobility and access

Task: Predict electric-bike share or duration by origin, destination, time, and availability. **Combine:** Weather, terrain, Census characteristics, and availability.
Benefit: Examine longer connections in transportation-constrained areas. **Limit:** Usage alone does not establish affordability or accessibility benefits.

6. Neighborhood connectivity through networks

Task: Cluster the station network and identify weak links or important hubs. **Combine:** Census characteristics and transit locations.
Benefit: Explore connections to employment and transit. **Limit:** Network centrality is not itself a measure of social need.

For network analysis, define $\mathbf{A_{ij}}$ **= number of trips from station i to station j**. Spectral clustering, matrix factorization, or graph embeddings can reveal travel communities. This connects to the course's linear-algebra work; forecasting offers a clearer supervised-learning evaluation.

Complementary public datasets

Source	Useful information	Proposed join
Census ACS API + TIGERweb [2, 3]	Poverty, income, vehicle availability, population, and commuting.	Map station coordinates to tracts; join GEOID. Use five-year estimates and retain margins of error.
NOAA hourly weather [4]	Temperature, precipitation, wind, and weather conditions.	Join observation time with a suitable nearby weather station.
MTA hourly subway ridership [5, 6]	Subway activity by station and hour.	Associate nearby bike and subway stations; align hourly aggregation. Separate datasets cover 2020-2024 and beginning 2025.
NYPD Motor Vehicle Collisions [7]	Dates, locations, cyclist injuries, and deaths.	Aggregate to tracts or spatial grid cells and periods. Police-reported motor-vehicle crashes do not capture all cycling injuries.
NYC Bicycle Routes [8]	Bicycle infrastructure and facility characteristics.	Calculate proximity and neighborhood coverage. Historical analysis needs infrastructure as it existed then.
NYCHA Development Data Book [9]	Public-housing characteristics and locations.	Calculate nearby station coverage and walking distances after checking location fields.
Citi Bike GBFS [1]	Real-time station information and availability.	Archive timestamped snapshots for empty/full-station analysis. Current data alone cannot reconstruct past availability.

Recommended initial combination

Citi Bike + NOAA + ACS + tract boundaries. Add transit proximity next. This is sufficient for a substantive final project without making several difficult datasets prerequisites.

Geographic and temporal consistency

Use ACS estimates with an appropriate reference period and retain geography vintage. A station's neighborhood is not necessarily a rider's home neighborhood. Historical station locations, Census boundaries, and infrastructure can change, so joins should retain their effective dates rather than use today's geography indiscriminately.

Recommended project design

Working title: Forecasting Citi Bike Demand and Identifying Transportation Access Gaps in New York City

Research question: How accurately can we predict station-level bike-share activity, and where do station coverage and model performance differ across neighborhoods with different transportation needs?

Use a complete recent 12-month period, if available, and reserve subsequent complete months for final testing. Use 2013 first to validate the historical import process; include it in the research analysis only for a defined longitudinal question.

Modeling table: one station per hour

• Separate departure and arrival counts; hour, weekday, holiday, and season.

• Prior-hour and prior-week activity; lagged rolling averages; weather.

• Station location, nearby transit, and neighborhood characteristics for analysis and an optionally evaluated feature set.

Include zero-trip hours for stations known to be operating. Otherwise, training data overrepresent busy periods. If historical operating status is unavailable, document inferred active periods rather than assuming every station existed throughout the dataset.

Model	Purpose
Same station, same hour, previous week	Practical forecasting baseline.
Poisson regression; negative-binomial if warranted	Interpretable count-model baseline.
Gradient-boosted trees	Capture nonlinear effects and interactions.

Evaluation and leakage prevention

Use chronological train, validation, and test periods. Report MAE and count deviance, with errors separated for busy stations, quieter stations, and neighborhood groups. A geographic holdout can test transfer to unfamiliar locations.

Operational forecasts can use only information available at prediction time. Use forecast weather rather than the next hour's observed weather, and avoid unpublished future transit counts.

Social-impact component

• **Coverage:** proximity to stations, especially for households without vehicles.

• **Observed activity:** actual and predicted usage.

• **Reliability:** empty/full-station minutes if availability snapshots are collected.

Low observed usage may reflect inadequate service and should not automatically reduce neighborhood priority. A manageable final deliverable is a forecasting model, access-gap map, and neighborhood performance comparison. Rebalancing or expansion recommendations are extensions.

Databricks deployment and file transfer

Databricks Connect supports local development while Spark processing runs remotely. It does not give cloud compute direct access to the Mac's `/Volumes/BigRaid` path. Files must first be uploaded. [10]

Approach	Processing location	Best use
Local Python/Spark + Parquet or Delta	Mac	Inventory, prototype cleaning, small experiments.
Databricks + uploaded files	Databricks compute	Course-project pilot with selected months.
Local IDE + Databricks Connect	Spark runs remotely	Local development after uploading data.
AWS S3 + Databricks on AWS	Databricks compute accessing S3	Full history and repeatable ingestion.

Recommended: prove the pipeline using a few complete months in a Databricks volume, then move to S3 if scale or course requirements justify it.

Pilot upload

Assuming `main` is available and you have creation privileges:

```
CREATE SCHEMA IF NOT EXISTS main.cuny622;
CREATE VOLUME IF NOT EXISTS main.cuny622.citibike_raw;
```

After configuring the CLI, run this on your Mac; adjust the local path if files are nested:

```
databricks fs cp \
  "/Volumes/BigRaid/RandyFiles/Cuny/622/FinalProject/201312-citibike-tripdata.csv" \
  "dbfs:/Volumes/main/cuny622/citibike_raw/201312-citibike-tripdata.csv"
```

CLI volume paths use `dbfs:/Volumes/...`; notebooks use `/Volumes/...`. The UI has a 5 GB per-file upload limit; both samples are below it. [11, 12]

AWS upload

Create a private bucket in the workspace's region. Preview the CSV-only upload:

```
aws s3 sync \
  "/Volumes/BigRaid/RandyFiles/Cuny/622/FinalProject/" \
  "s3://YOUR-BUCKET/citibike/raw/incoming/" \
  --exclude "*" --include "*.csv" \
  --no-follow-symlinks --dryrun
```

Review the preview, then remove `--dryrun` to upload. ZIPs are excluded. Connect S3 using a Unity Catalog storage credential backed by an IAM role and an external location. Keep credentials out of notebooks. These are proposed commands; no upload or deployment was performed. [13, 14]

Import pipeline: raw files to usable trips

1. Inventory the Mac folder

Create a manifest with relative path, size, checksum, header/schema family, row count, actual date range, and nominal month/part. Identify archives and extracted copies to avoid duplicate ingestion. Verify every part of each selected month. Use actual timestamps for trip dates; filenames describe source releases.

2. Bronze: preserve raw records

Classify files by actual headers and keep historical and modern schemas separate. Read raw fields as strings; retain source filename, checksum, ingestion time, and schema version. Sanitize legacy column names containing spaces while keeping the original-name mapping. Retain malformed records for inspection.

Make reruns idempotent through the manifest or ingestion checkpoints. Use batch ingestion for the first months. For recurring arrivals, use Auto Loader with separate schema locations and checkpoints for incompatible families. [15]

3. Silver: normalize the schemas

Historical field	Modern field	Unified field
starttime	started_at	started_at
stoptime	ended_at	ended_at
start station id	start_station_id	start_station_id
end station id	end_station_id	end_station_id
usertype	member_casual	membership_group; retain original category
Not supplied	rideable_type	rideable_type; null if unavailable
birth year; gender	Not supplied	Optional historical fields

Keep station IDs as strings: values such as `6322.01` are identifiers. Build station-location history rather than assuming IDs remain comparable across eras. Historical Subscriber/Customer categories can be grouped for comparison, but preserve their original definitions.

Parse second and fractional-second timestamps. Preserve local wall time, validate the source timezone convention, and document daylight-saving ambiguity when converting to UTC.

Flag invalid coordinates, missing endpoints, implausible durations, and suspect demographic values. Keep raw records and document which flags exclude a trip from each analysis.

Features, validation, and execution sequence

4. Gold: modeling and reporting tables

- `station_hour_activity` and `station_location_history`
- `station_tract_crosswalk` and `tract_access_indicators`
- `weather_hourly` and `station_hour_features`

Spatially join the small station-location table to tract polygons, then attach the crosswalk to trips. Preserve geography vintage and use boundary crosswalks when tracts change. Keep feature generation distributed and move only appropriately sized aggregates into local modeling. Record model parameters, metrics, and data versions with MLflow.

5. Validate before training

Reconcile imported rows with the manifest, verify month completeness, check duplicate ride IDs and overlapping downloads, and inspect station-hour totals. Historical deduplication needs a separate policy because those files lack modern ride IDs.

Start with modest compute, enable automatic termination where available, and scale after measuring ingestion and feature-generation performance.

Final-project sequence

Stage	Concrete result
1. Inventory and pilot import	Verified files and normalized sample tables.
2. Assemble a complete recent period	Station-hour counts, weather, tract crosswalk, and ACS indicators.
3. Establish baselines	Previous-week forecast and count regression.
4. Train and evaluate ML	Boosted-tree model with chronological testing.
5. Analyze social impact	Coverage maps and neighborhood performance comparisons.
6. Optional extension	Availability monitoring, rebalancing simulation, or candidate-station analysis.

Initial scope: hourly activity forecasting plus an access-gap analysis. This provides measurable predictive results and a credible social purpose. Actual service-failure prediction requires availability data.

Evidence and implementation status

Sample statistics came from direct inspection of 201312-citibike-tripdata.csv and 202608-citibike-tripdata_1.csv. External sources were reviewed for the original analysis on September 30, 2026. Proposed models, joins, commands, and pipeline steps have not yet been implemented or evaluated on the full collection.

Sources and download links

Numbers in the report refer to the official sources below. Links are clickable. Sample statistics are derived from the attached files, rather than the external sources.

[1] Citi Bike system data, historical formats, split files, and GBFS
https://citibikenyc.com/system-data

[2] Census American Community Survey API
https://www.census.gov/programs-surveys/acs/data/data-via-api.html

[3] Census TIGERweb boundary services
https://tigerweb.geo.census.gov/tigerwebmain/TIGERweb_restmapservice.html

[4] NOAA Global Hourly / Integrated Surface Database
https://www.ncei.noaa.gov/products/land-based-station/integrated-surface-database

[5] MTA Subway Hourly Ridership: Beginning 2025
https://data.ny.gov/Transportation/MTA-Subway-Hourly-Ridership-Beginning-2025/5wq4-mkjj

[6] MTA Subway Hourly Ridership: 2020-2024
https://data.ny.gov/Transportation/MTA-Subway-Hourly-Ridership-2020-2024/wujg-7c2s

[7] NYC Motor Vehicle Collisions - Crashes
https://data.cityofnewyork.us/Public-Safety/Motor-Vehicle-Collisions-Crashes/h9gi-nx95

[8] New York City Bike Routes
https://data.cityofnewyork.us/dataset/New-York-City-Bike-Routes/mzxg-pwib

[9] NYCHA Development Data Book
https://data.cityofnewyork.us/Housing-Development/NYCHA-Development-Data-Book/evjd-dqpz

[10] Databricks Connect
https://docs.databricks.com/aws/en/dev-tools/databricks-connect/

[11] Databricks CLI fs commands
https://docs.databricks.com/aws/en/dev-tools/cli/reference/fs-commands

[12] Files in Unity Catalog volumes
https://docs.databricks.com/aws/en/volumes/volume-files

[13] AWS CLI s3 sync
https://docs.aws.amazon.com/cli/latest/reference/s3/sync.html

[14] Connect Databricks to an AWS S3 external location
https://docs.databricks.com/aws/en/connect/unity-catalog/cloud-storage/s3

[15] Databricks Auto Loader
https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/